



Breaking the Monolith:

Implementing Airflow 3.x Remote Execution for Multi-Team Environments



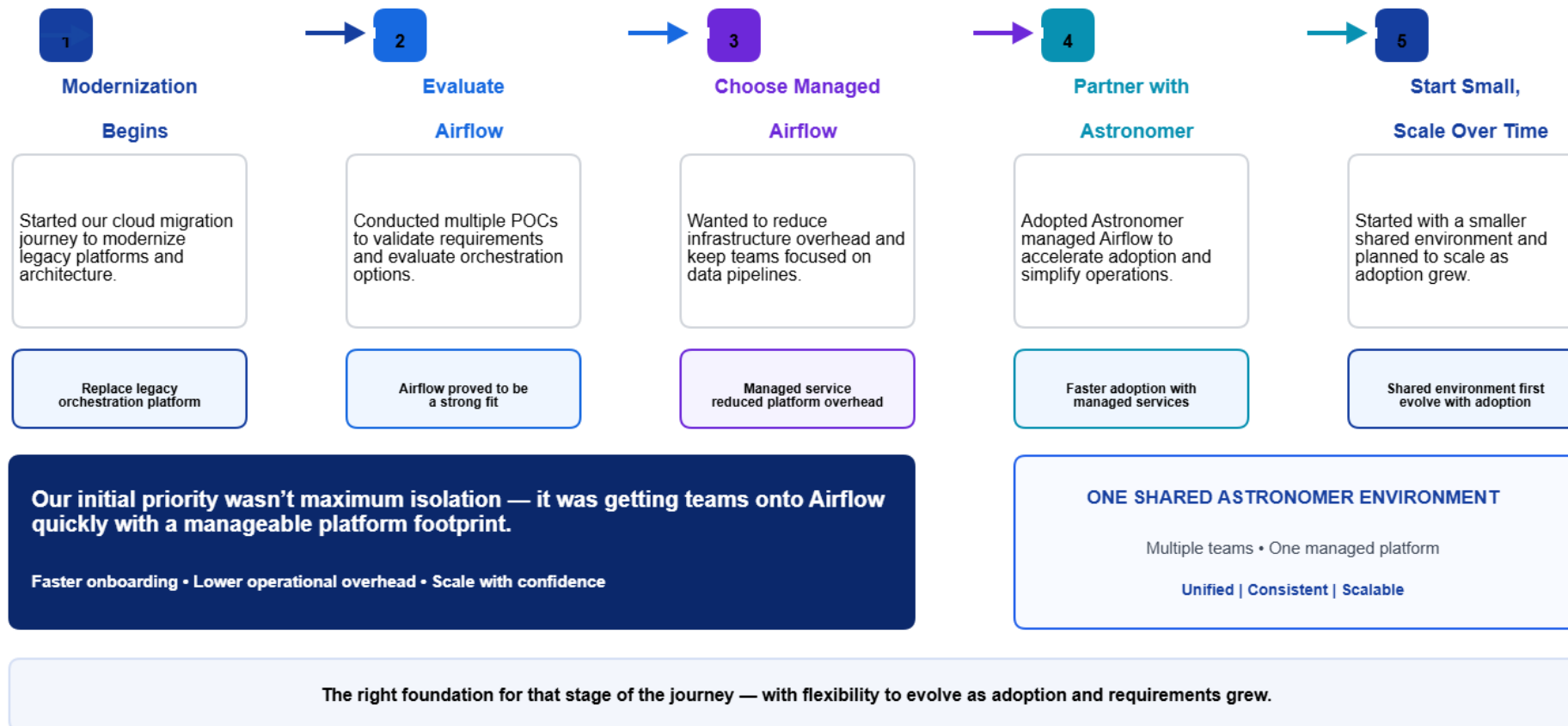
Kowsy Narayan, Ontario Teachers'

About Ontario Teachers'



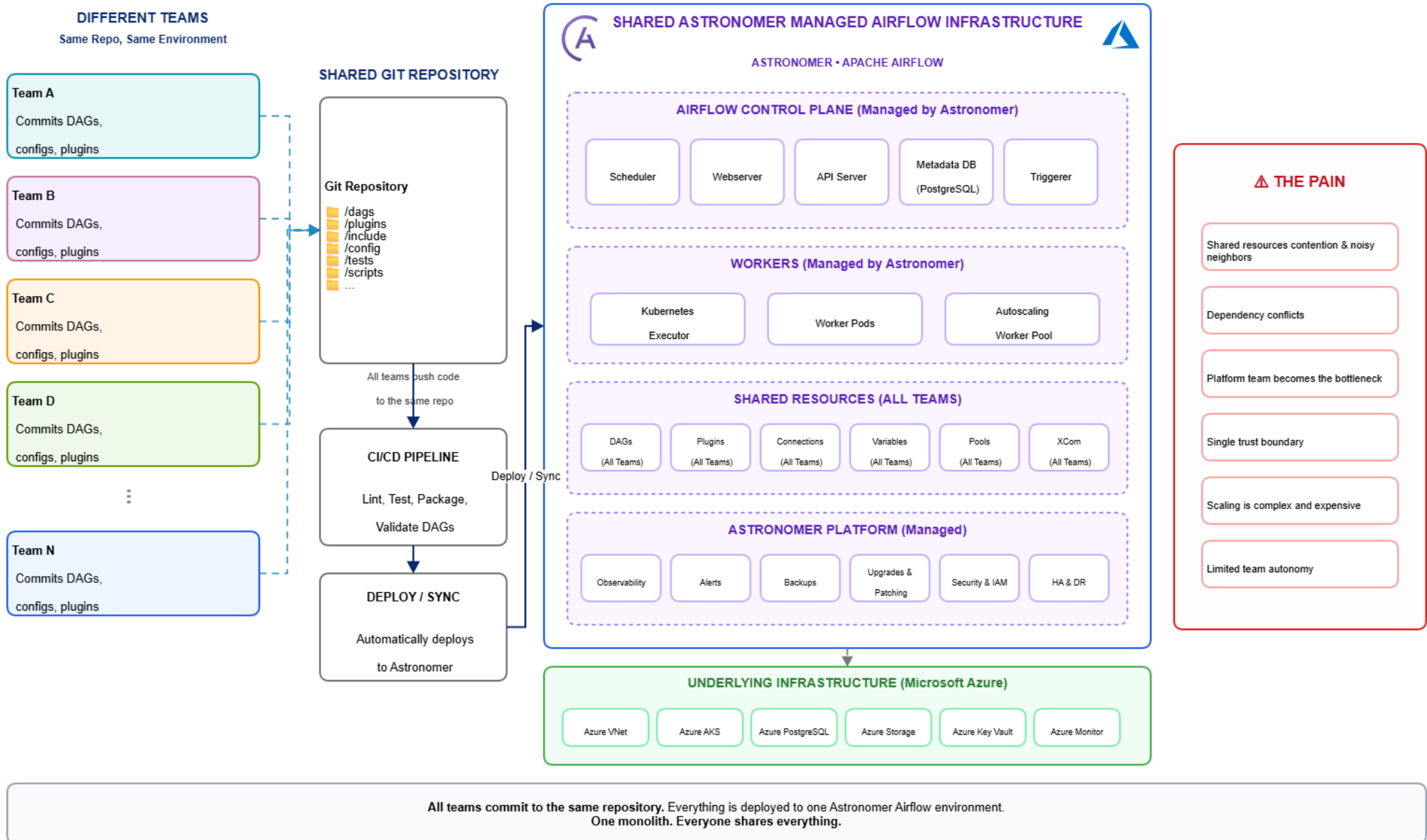
Our Journey

From legacy orchestration to a managed, scalable foundation



The Monolith We Started With

All teams commit to the same repo that deploys to one shared Airflow infrastructure on Astronomer



Pain Points



Platform bottlenecks

Central platform team becomes the queue for every workload change



Security boundaries

Shared runners blur the line between teams and sensitive data



Resource contention

Jobs compete for the same compute, slowing everyone down



Release coordination

Teams must sync schedules instead of shipping independently



Custom CI/CD pipeline

Bespoke pipelines per team drive duplicated maintenance effort

Design Goals



Single control plane

One place to define, run and observe every workload



Team autonomy

Teams ship and deploy code on their own schedule without central bottlenecks



Execution isolation

Every task runs in its own sandbox with no shared blast radius



Scalable governance

Policy and controls applied consistently as usage grows

Airflow 3.x Changes the Execution Model

Centralized orchestration. Distributed execution



Edge Execution

Tasks run on remote edge workers that pull work over HTTPS, with a native UI for managing them



Remote Agents

Lightweight agents in your own Kubernetes cluster, outbound-only, no inbound firewall exposure



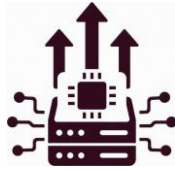
Centralized Observability

Live remote task logs stream into one UI for unified monitoring and troubleshooting

Remote Execution use cases



**Secure access to
sensitive data**



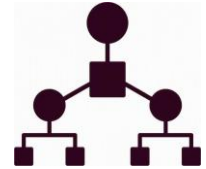
**Bring your own
compute**



**Design your
execution plane**

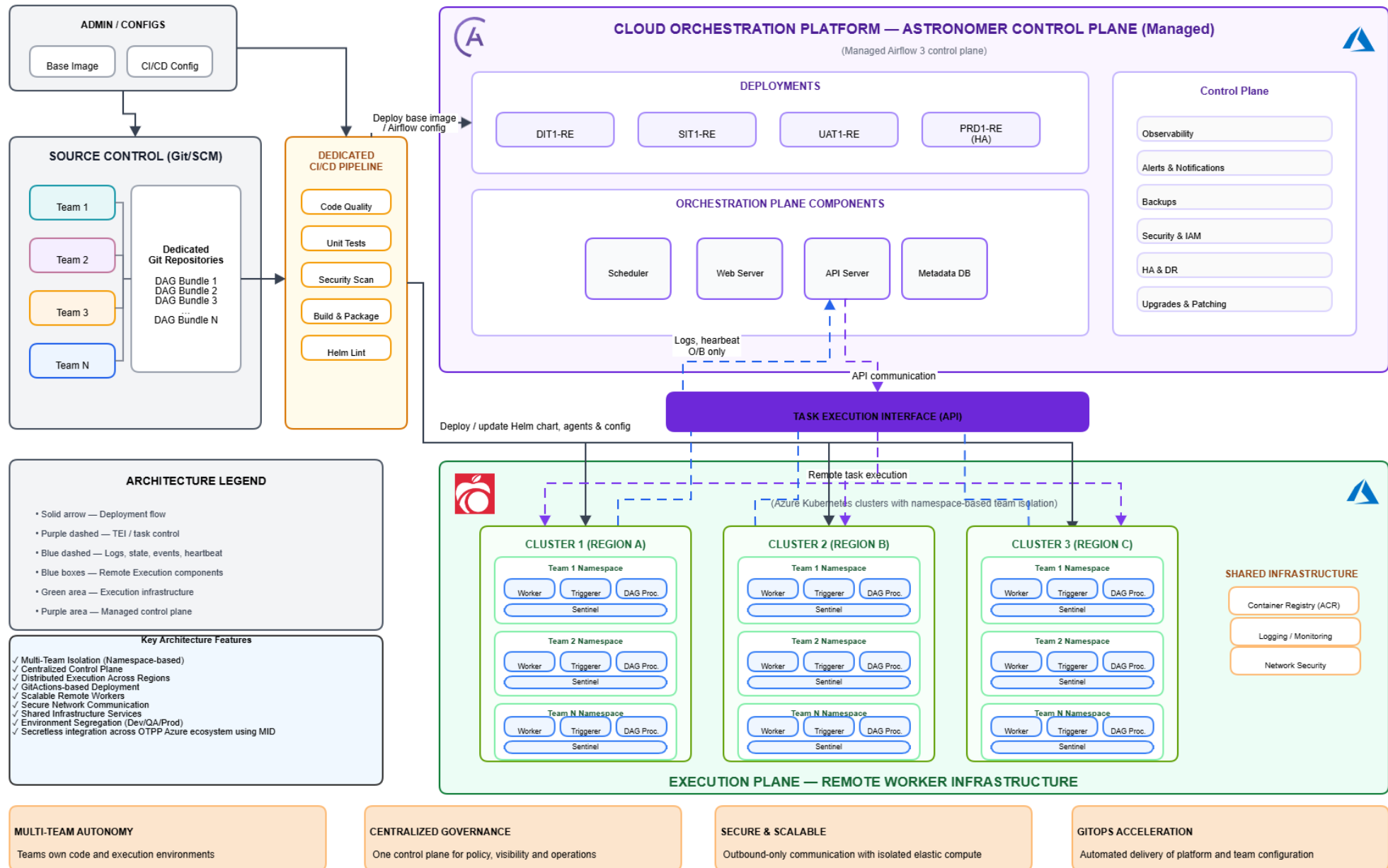


**Complete isolation
of executions**



**Simplified network
architecture**

Target State - Centralized Orchestration, Distributed Remote Execution



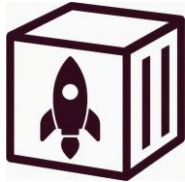
Upgrade Considerations

Remote execution changes the operating model, not just the architecture



Airflow runtime image

Track base Airflow version changes before each upgrade



Astro RE runtime image

Remote execution runtime must stay compatible with the platform



Astro Sentinel image

Monitoring agent version kept in step with the runtime



Helm Chart

Chart version drives the rollout of every component above

What we achieved?



Standardized base image

One centrally managed Docker base image for every workload



Standardized Helm charts

Charts aligned to OTPP security standards out of the box



Standardized onboarding

A single, repeatable path for teams joining the platform



Standardized CI/CD

Automation teams inherit instead of rebuilding each time



Centralized control plane

One place to run and monitor every execution

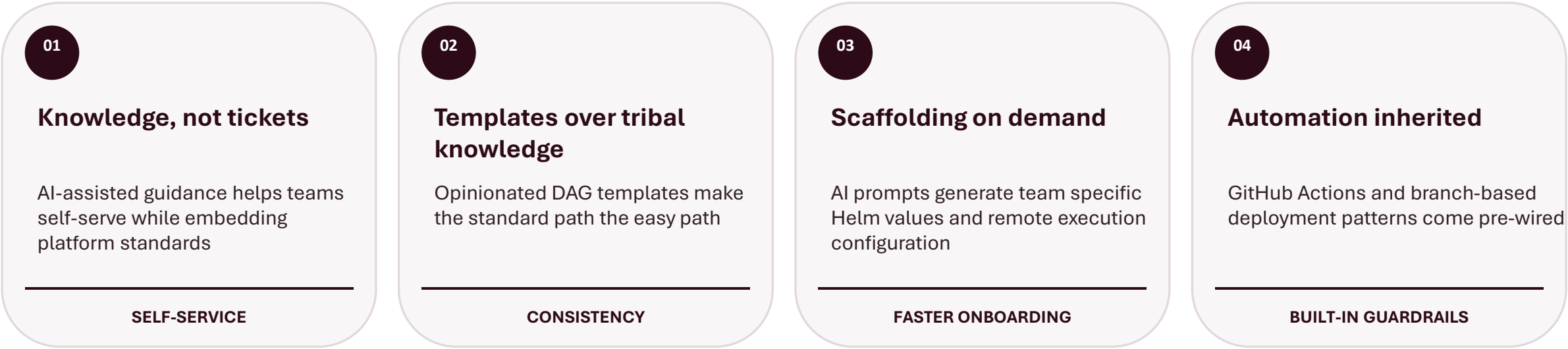


Streamlined upgrades

Control and execution planes upgrade with minimal disruption

The Next Hill to Climb

Remote execution removed the infrastructure bottleneck. The next challenge is enabling every team to operate independently — without becoming Airflow platform experts.



The platform scales when our knowledge scales with it.



Questions?

Linkedin:

